Time and Space Complexity

A beginner-to-advanced guide for Java and coding interviews

Learning goal: Learn how to estimate how an algorithm's running time and extra memory grow as the input size grows. You do not need to calculate the exact number of CPU seconds.

1. What is input size?

Input size is commonly written as **n**. For an array, n is the number of elements. For a string, n is its length. For a matrix, we may use rows r and columns c, or n for a square n × n matrix. Complexity describes how the work or memory changes as input size grows.

```
int[] nums = {4, 8, 2, 9};
// Here n = 4
```

2. Time complexity: how much work?

Time complexity describes how the number of basic operations grows with input size. It is not a promise of exact seconds: hardware, JVM, input values, and implementation affect actual runtime.

Example: one pass through an array

```
for (int i = 0; i < n; i++) {
    System.out.println(nums[i]);
}
```

The loop runs about n times, so its time complexity is O(n). If n doubles, the work roughly doubles.

3. Big O, Big Omega, and Big Theta

Notation	Meaning (informally)	Typical use
O(f(n))	Upper bound; often used for worst-case growth	Algorithm interview analysis
Ω(f(n))	Lower bound	Minimum growth guarantee
Θ(f(n))	Tight bound: both upper and lower growth match	Precise asymptotic classification

Big O is not automatically the worst case in every mathematical context. In interviews, people often use it to report worst-case time, so state the case you are analyzing.

4. Common time complexities

Complexity	Name	Example / intuition
O(1)	Constant	Read nums[0]; work does not grow with n.
O(log n)	Logarithmic	Binary search; repeatedly halve the search range.
O(n)	Linear	Scan every array element once.
O(n log n)	Linearithmic	Efficient comparison sorts such as merge sort.
O(n²)	Quadratic	Two nested loops each visiting n items.
O(n³)	Cubic	Three nested loops over n items each.
O(2■)	Exponential	Some brute-force subset/choice algorithms.
O(n!)	Factorial	Enumerating all permutations in a brute-force approach.

5. How to analyze loops

A single loop that runs n times is O(n). Two sequential loops are O(n) + O(n) = O(n), because constants are ignored. Two nested loops that each run n times are O(n × n) = O(n²).

```
// O(n)
for (int i = 0; i < n; i++) { work(); }

// O(n^2)
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        work();
    }
}

// O(log n)
while (n > 1) {
    n /= 2;
}
```

Important: nested loops do not always mean O(n²). Analyze the actual number of iterations. For example, if the inner loop runs only a fixed 5 times, total work is O(n).

6. Drop constants and lower-order terms

If T(n) = 3n² + 10n + 50, its asymptotic complexity is O(n²). For large n, the n² term dominates. We drop constant multipliers and smaller-order terms when describing growth.

```
O(5n)       → O(n)
O(n + 100)  → O(n)
O(n^2 + n)  → O(n^2)
```

7. Space complexity

Space complexity describes how memory usage grows with input size. Auxiliary space means extra memory used by the algorithm, excluding the input itself unless the problem or interviewer asks for total space.

```
// O(1) auxiliary space
int sum = 0;
for (int x : nums) sum += x;

// O(n) auxiliary space
int[] copy = new int[n];
```

A few variables usually use O(1) auxiliary space. An extra array of n elements uses O(n). A recursive algorithm may also use call-stack memory.

8. Recursion and call-stack space

Each recursive call creates a stack frame containing method state and parameters. If the maximum number of active calls is proportional to n, stack space is O(n). If recursion depth is proportional to log n, stack space is O(log n). Analyze maximum depth, not total number of calls.

```
// Linear depth: O(n) stack
void f(int n) {
    if (n <= 0) return;
    f(n - 1);
}

// Halving depth: O(log n) stack
void g(int n) {
    if (n <= 1) return;
    g(n / 2);
}
```

9. Divide and conquer: Merge Sort

Merge Sort divides the array into halves until each part has one element, then merges sorted parts. There are about log n levels of division, and merging across each level processes a total of n elements. Therefore time is O(n log n). The temporary merge arrays use O(n) auxiliary space; recursive stack depth is O(log n), so the overall auxiliary space remains O(n).

```
T(n) = 2T(n/2) + O(n)

Time:  O(n log n)
Space: O(n)
```

10. Tree DFS example: Sum Root to Leaf Numbers

For a binary tree with n nodes, a DFS that visits each node once has O(n) time. The recursive stack depends on tree height h, so stack space is O(h). In a balanced tree h is O(log n); in a skewed tree h can be O(n). For this problem, each path number is built in constant time per node, so the total traversal time is O(n).

```
private int dfs(TreeNode root, int currentNumber) {
    if (root == null) return 0;

    currentNumber = currentNumber * 10 + root.val;

    if (root.left == null && root.right == null) {
        return currentNumber;
    }

    return dfs(root.left, currentNumber)
         + dfs(root.right, currentNumber);
}

Time:  O(n)
Space: O(h) recursion stack
```

11. Best, average, and worst cases

Best case is the most favorable input; worst case is the least favorable; average case estimates typical behavior under an assumed input distribution. Example: linear search is best-case O(1) if the first element matches, and worst-case O(n) if the target is last or absent.

12. Practical analysis checklist

1. Define the input size: n, rows × columns, number of nodes, etc.
2. Count how many times the key operation runs.
3. For loops, inspect bounds and how variables change.
4. For recursion, write the recurrence and measure maximum call depth.
5. For extra memory, count arrays, collections, and stack frames.
6. Drop constants and lower-order terms.
7. State whether the result is best, average, or worst case when relevant.

13. Quick practice

Code / scenario	Expected complexity
Access nums[3]	Time O(1)
Scan an array once	Time O(n)
Compare every pair using two full nested loops	Time $O(n^2)$
Binary search in a sorted array	Time O(log n)

Code / scenario	Expected complexity
Create a new array of length n	Auxiliary space O(n)
DFS visits every tree node once	Time O(n); stack O(h)

Final memory trick: Time asks “How much work as input grows?” Space asks “How much extra memory as input grows?” Analyze growth, not exact seconds or bytes.